CSC 212: Data Structures and Abstractions

12: Priority Queues, Binary Heaps

Prof. Marco Alvarez

Department of Computer Science and Statistics
University of Rhode Island

Fall 2026



Priority queues

- A **priority queue** stores elements, each with a **key** (its priority)
  - **Max-priority queue**: the largest key is served first
  - **Min-priority queue**: the smallest key is served first
- Applications
  - schedulers (operating systems, inference servers), event-driven simulation
  - shortest paths and minimum spanning trees on graphs
  - merging sorted streams, top-*k* queries

operation	what it does
push(x, p)	add an element with priority p
top()	the element with the highest priority
pop()	remove that element
size()	return the number of elements
empty()	return true if the priority queue is empty, and false otherwise



Q12-1

- A max-priority queue is used where each element's key is its *arrival number*: 0, 1, 2, ... What does it behave like?

A. A queue: oldest first
B. A stack: newest first
C. A sorted array
D. It serves elements in random order
E. None of the above

Answer

- B) A stack: newest first
  - the newest element has the largest arrival number, so it has the highest priority

std::priority_queue

- A **max-priority** queue by default
  - `std::pair` compares by `first`, then by `second`

```
std::priority_queue<std::pair<int, std::string>> pq;
pq.push({2, "summarize 500 PDFs"});
pq.push({9, "chat: hello"});
pq.push({5, "autocomplete"});
std::cout << pq.top().second << "\n"; // chat: hello
pq.pop();
std::cout << pq.top().second << "\n"; // autocomplete
```

- A **min-priority** queue takes two more template arguments

```
std::priority_queue<int, std::vector<int>, std::greater<int>> mq;
```


Q12-2

- What does this program print?

```
std::priority_queue<std::pair<int, std::string>> pq;
pq.push({3, "Job 1"});
pq.push({1, "Job 2"});
pq.push({5, "Job 3"});
pq.pop();
pq.push({2, "Job 4"});
pq.pop();
pq.push({7, "Job 5"});
pq.pop();
pq.pop();
pq.push({7, "Job 6"});
pq.push({7, "Job 7"});
while (!pq.empty()) {
    std::cout << pq.top().second << "\n";
    pq.pop();
}
```

A. Job 6, Job 7, Job 2
B. Job 7, Job 6, Job 2
C. Job 7, Job 6, Job 4
D. Job 2, Job 6, Job 7
E. None of the above

Answer

- B) Job 7, Job 6, Job 2

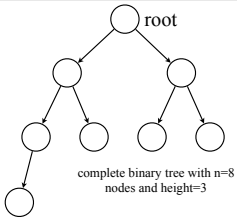
[trace the code step by step. The two 7s tie on `first`, so `second` decides]

Two obvious designs

- Unsorted vector
  - push appends: $\Theta(1)$ amortized cost
  - pop scans for the max: $\Theta(n)$ cost
- Sorted vector
  - push shifts to make room: $\Theta(n)$ cost
    - requires finding position for insertion
  - pop takes the last: $\Theta(1)$ cost
- Each design is cheap at one end and expensive at the other

Complete binary trees

- Binary tree
  - each **node** has at most two children, a left child and a right child
- Complete binary tree
  - every level is full, except possibly the last, which is filled **from the left**
- Level d, starting from level 0, holds at most 2^d nodes



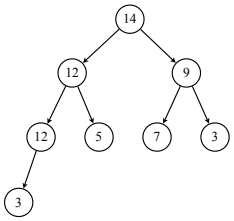
complete binary tree with n=8 nodes and height=3

The **height of a tree** is defined as the number of edges on the longest path from the root to a leaf (terminal) node.

The height h of a complete binary tree with n nodes is $\lfloor \log_2 n \rfloor$

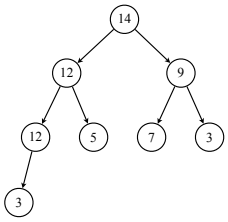
The heap property

- A **binary max-heap** is a complete binary tree in which every node is $\geq$ its children
  - **shape property**: complete binary tree
  - **heap property**: parent $\geq$ child, on every edge
  - consequences
    - **the max is at the root:** $\Theta(1)$
    - every **root-to-leaf** path is sorted, decreasing
- A **binary min-heap** flips the inequality: every node is $\leq$ its children



Array representation

- A binary heap can be represented as an array
  - **root** is at index 0
  - **last element** is at index $n - 1$
- For any node at index i:
  - **left child** is at index $2i + 1$
  - **right child** is at index $2i + 2$
  - **parent** is at index $(i - 1)$ / 2 (integer division)



0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
14	12	9	12	5	7	3	3								

n=8, capacity=16

Q12-3

- A max-heap holds 15 distinct keys at indices 0 to 14: a full tree of height 3. Where can the **smallest** key be?

A. Always at index 14, the last position
B. At any index except 0
C. At index 1 or 2
D. At any of the indices 7 to 14
E. None of the above

Answer

- D) At any of the indices 7 to 14
  - ✓ a node with a child is larger than that child, so the smallest key has **no children**: it is a **leaf**
  - ✓ the 8 leaves are the second half of the array

A binary max-heap class

```
class MaxHeap {
    private:
        std::vector<int> m_data; // the tree, level by level

        static size_t parent(size_t i) { return (i - 1) / 2; }
        static size_t left(size_t i) { return 2 * i + 1; }
        static size_t right(size_t i) { return 2 * i + 2; }

        void sift_up(size_t i);
        void sift_down(size_t i);

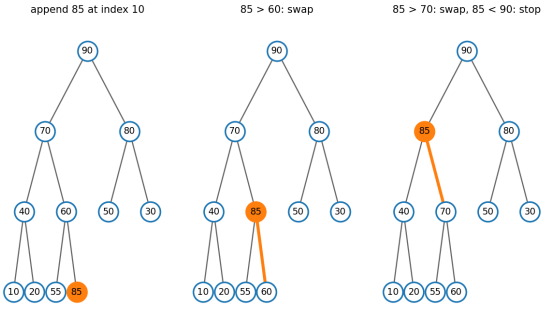
    public:
        MaxHeap() {}
        MaxHeap(const std::vector<int>& v); // build-heap

        void push(int x);
        void pop();
        const int& top() const;
        size_t size() const { return m_data.size(); }
        bool empty() const { return m_data.empty(); }
};
```


`push`: append, then sift up

- Algorithm (min-heap is analogous)
  1. **append at index *n***: the shape stays complete
     - the heap property can now fail on one edge only: new node vs. its parent
  2. **sift up**: while the node is larger than its parent, swap them
     - the violation moves up one level per swap, and stops at the root at the latest
- Time complexity
  - ✓ how many swaps are necessary in the worst case? $\Theta(\log n)$

https://visualgo.net/en/heap



`push` and `sift_up`

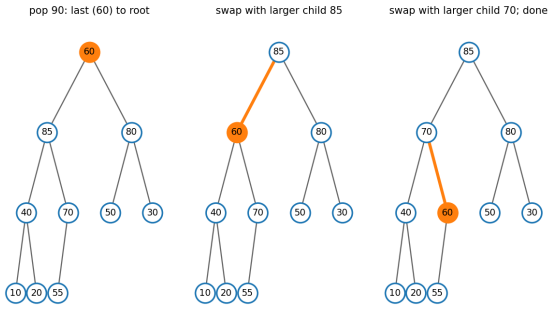
```
void MaxHeap::push(int x) {
    m_data.push_back(x); // still complete tree
    sift_up(m_data.size() - 1); // repair one path
}


void MaxHeap::sift_up(size_t i) {
    while (i > 0 && m_data[i] > m_data[parent(i)]) {
        std::swap(m_data[i], m_data[parent(i)]);
        i = parent(i);
    }
}
```


`pop`: move last to root, then sift down

- Algorithm (min-heap is analogous)
  1. **move the last element to the root, shrink by one**: the shape stays complete
     - the max is at index 0, but removing the root leaves a hole in the wrong place
  2. **sift down**: while the node is smaller than a child, swap it with the **larger** child
- Time complexity
  - how many swaps are necessary in the worst case? $\Theta(\log n)$

https://visualgo.net/en/heap



`pop` and `sift_down`

```
void MaxHeap::pop() {
    if (empty())
        throw std::underflow_error("pop on empty heap");
    m_data[0] = m_data.back(); // the last element fills the root
    m_data.pop_back();
    if (! empty()) sift_down(0);
}

void MaxHeap::sift_down(size_t i) {
    size_t n = m_data.size();
    while (true) {
        size_t big = i; // the largest of i and its children
        if (left(i) < n && m_data[left(i)] > m_data[big]) big = left(i);
        if (right(i) < n && m_data[right(i)] > m_data[big]) big = right(i);
        if (big == i) return; // i is >= both children: done
        std::swap(m_data[i], m_data[big]);
        i = big;
    }
}
```

Q12-4

- Push 65 into the heap [85, 70, 80, 40, 60, 50, 30, 10, 20, 55]. What is the array after `push` returns?

A. [85, 70, 80, 40, 60, 50, 30, 10, 20, 55, 65]
B. [85, 70, 80, 40, 65, 50, 30, 10, 20, 55, 60]
C. [85, 65, 80, 40, 70, 50, 30, 10, 20, 55, 60]
D. [85, 70, 80, 65, 60, 50, 30, 10, 20, 55, 40]
E. None of the above

Answer

- B) [85, 70, 80, 40, 65, 50, 30, 10, 20, 55, 60]
  - ✓ append 65 at index 10
  - ✓ only the path from index 10 to the root was looked at: 2 comparisons in a heap of 11
    - the other 8 elements were never touched

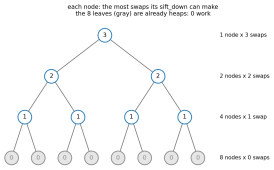
The cost is the height

- `push` and `pop` each walk one root-to-leaf path
  - ✓ a constant amount of work per level
- The height is $\lfloor \log_2 n \rfloor$: both are $\Theta(\log n)$ in the worst case

Method	Unsorted Array	Sorted Array	Binary Heap
Push	O(1)	O(n)	O(log n)
Pop	O(n)	O(1)	O(log n)
Top	O(n)	O(1)	O(1)
Size	O(1)	O(1)	O(1)
IsEmpty	O(1)	O(1)	O(1)

Building a heap from a vector of *n* elements

- First idea: ***n* pushes**
  - ✓ the element at depth *d* may climb *d* levels
  - ✓ worst case (keys arrive increasing): $\Theta(n \log n)$
- Better idea: **bottom-up**
  - ✓ the leaves are already one-node heaps
  - ✓ for each internal node, from the last one back to the root: `sift_down`



The last internal node is parent(n − 1) = n/2 − 1; everything after it is a leaf

```
MaxHeap::MaxHeap(const std::vector<int>& v) {
    m_data = v;
    size_t n = m_data.size();
    for (size_t i = n / 2; i > 0; --i)
        sift_down(i - 1);
}
```

https://visualgo.net/en/heap

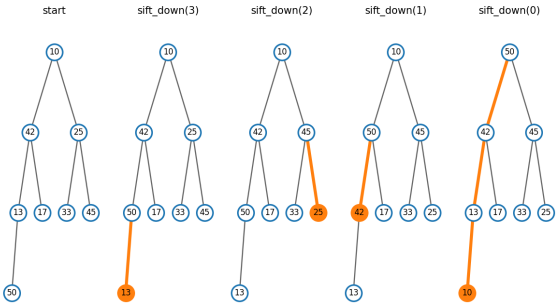
Q12-5

- Run bottom-up `build-heap` on [10, 42, 25, 13, 17, 33, 45, 50]. What is the resulting array?

A. [50, 45, 42, 33, 25, 17, 13, 10]
B. [50, 45, 42, 17, 13, 25, 33, 10]
C. [50, 42, 45, 13, 17, 33, 25, 10]
D. [50, 42, 45, 10, 17, 33, 25, 13]
E. None of the above

Answer

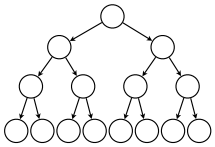
- C) [50, 42, 45, 13, 17, 33, 25, 10]



Why `build-heap` is linear?

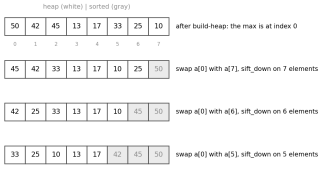
- Sift down from a node costs at most its height: the levels below it
  - ✓ half the nodes are leaves: height 0, no work
  - ✓ a quarter have height 1, an eighth height 2, ...
  - ✓ only the root has height h

$$T(n) = 1(h) + 2(h-1) + 4(h-2) + \ldots + 2^h(0)$$
$$= \sum_{i=0}^{h} 2^i(h-i)$$
$$= h\sum_{i=0}^{h} 2^i - \sum_{i=0}^{h} i2^i = \ldots\ \ldots\ \ldots = \Theta(n)$$



Heapsort

- Algorithm (in-place)
  1. build a max-heap in the array: $\Theta(n)$
  2. swap the root (the max) with the last element of the heap: the max is in its final place
  3. shrink the heap by one, `sift_down(0)`: $\Theta(\log n)$
  4. repeat steps 2-3 until one element is left



heap_sort

```
void heap_sort(std::vector<int>& a) {
    size_t n = a.size();
    if (n < 2) return;

    for (size_t i = n / 2 ; i > 0 ; --i) // build-heap
        sift_down(a, i - 1, n);

    for (size_t end = n - 1 ; end > 0 ; --end) {
        std::swap(a[0], a[end]); // move the max
        sift_down(a, 0, end); // repair the heap
    }
}
```


Exercises

- Numbers arrive one at a time; after each one, report the *k*-th largest seen so far
  - example: *k = 3*, stream "4 5 8 2 3 5 10 9 4"
  - output: "- - 4 4 4 5 5 8 8"
  - hint: keep only *k* numbers. Which of them do you need to see, and which kind of heap shows it? What is the cost per number, in terms of *k*?
- Given *n* points in the plane and *k*, return the *k* points closest to the origin. Reach $\Theta(n + k \log n)$
  - example: *k = 2*, points "(1,3) (-2,2) (5,8) (0,1)"
  - output: "(0,1) (-2,2)"
  - hint: build_heap, do not push
  - hint: compare $x^2 + y^2$; no *sqrt* is needed

Exercises

- Write `bool is_max_heap(const std::vector<int>& a)` in $\Theta(n)$, then answer:
  - is a vector sorted in **decreasing** order always a max-heap?
  - is a vector sorted in **increasing** order ever a max-heap?
- Start from the max-heap [30, 12, 9, 12, 5, 7, 3, 3]
  - push 20, then 1, then 50
  - then pop three times
  - draw the tree and write the array after every operation

Exercises

- Given a vector and a window size *k*, return the **maximum** of every window of *k* consecutive elements
  - example: *k = 4*, values 1, 13, -1, -3, 5, 3, 6, 7
  - output: 13, 13, 5, 6, 7
  - hint: solve it with a max-heap of (value, index) pairs
    - a heap cannot remove an element from the middle
    - leave old entries in, and discard one only when it reaches the top (lazy deletion)
- This is max pooling in neural networks: a pooling layer slides a window over its input and keeps only the largest value in each window

Sliding Window Maximum — Lazy Deletion Max-Heap

k = 4, values = {1, 13, −1, −3, 5, 3, 6, 7}

Heap stores **(val, idx)** pairs in max-heap array order. Expiry threshold at step i: index j is expired when $j \leq i - k$.

i	val	Action	Heap	Window Max
0	1	push	{(1,0)}	—
1	13	push	{(13,1),(1,0)}	—
2	−1	push	{(13,1),(1,0),(−1,2)}	—
3	−3	push; top (13,1) valid	{(13,1),(1,0),(−1,2),(−3,3)}	**13**
4	5	push; top (13,1) valid	{(13,1),(5,4),(−1,2),(−3,3),(1,0)}	**13**
5	3	push; pop expired (13,1)	{(5,4),(1,0),(3,5),(−3,3),(−1,2)}	**5**
6	6	push; top (6,6) valid	{(6,6),(1,0),(5,4),(−3,3),(−1,2),(3,5)}	**6**
7	7	push; top (7,7) valid	{(7,7),(1,0),(6,6),(−3,3),(−1,2),(3,5),(5,4)}	**7**